

Ejercicio en Vivo: Analizador Financiero Personal con Tests

Pruebas Unitarias y TDD con `pytest`

Curso de Programación en Python – Semana 5

29 de junio de 2026

1. Introducción

Este ejercicio práctico guiado sirve para consolidar los fundamentos de `pytest`, desarrollo guiado por pruebas (TDD) y análisis de cobertura (`pytest-cov`) resolviendo en vivo un problema análogo a la tarea semanal.

Aprenderás a estructurar pruebas unitarias robustas evaluando tres escenarios críticos:

- **Casos Normales:** Flujos estándar con entradas típicas y válidas.
- **Casos Límite (Edge Cases):** Valores extremos como saldos en cero, montos mínimos o transacciones en los bordes de la lógica de negocio.
- **Manejo de Excepciones:** Validación de que el sistema robustamente lance errores controlados frente a entradas absurdas o prohibidas.

2. Objetivos de Aprendizaje

- Implementar la estructura profesional AAA (*Arrange, Act, Assert*) utilizando el framework `pytest`.
- Aplicar la metodología TDD (*Red-Green-Refactor*) para el diseño de lógica financiera elemental.
- Emplear manejadores de contexto como `pytest.raises` con filtros de coincidencia de mensajes.
- Generar e interpretar reportes detallados de cobertura de código.

3. Enunciado del Proyecto: El Analizador Financiero

Diseñaremos e implementaremos un módulo llamado `finanzas.py` que actuará como el motor lógico de una billetera virtual y analizador de presupuestos personales. Posteriormente, crearemos el archivo de pruebas correspondiente denominado `test_finanzas.py`.

3.1. Requisitos Funcionales y Métodos a Implementar

El archivo `finanzas.py` debe incluir tres conjuntos clave de operaciones de complejidad incremental:

3.1.1. 1. Gestión de Cuenta y Balance

- `deposito(saldo_actual, monto)`: Retorna el nuevo saldo tras sumar el monto. Debe lanzar un `ValueError` con el mensaje *El monto a depositar debe ser positivo* si el monto es menor o igual a cero.
- `retiro(saldo_actual, monto)`: Retorna el nuevo saldo tras deducir el monto. Debe lanzar un `ValueError` con el mensaje *Fondos insuficientes* si el retiro excede el saldo. Además, lanzará un `ValueError` con el mensaje *El monto a retirar debe ser positivo* si el monto es menor o igual a cero. `raise ValueError("Mensaje")`

3.1.2. 2. Proyecciones Financieras Avanzadas

- `calcular_interes_compuesto(capital, tasa, periodos)`: Calcula el monto final acumulado usando la fórmula financiera:

$$A = P \times (1 + r)^t$$

Donde P es el capital, r la tasa de interés y t los períodos. Lanza un `ValueError` si el capital es negativo o si la tasa es menor que -1 (pérdida total superior al 100 %).

- `es_cuenta_premium(saldo, antiguedad_meses)`: Retorna `True` si el saldo es mayor o igual a \$10,000 o si el cliente posee una antigüedad de al menos 24 meses siempre que su saldo actual no sea negativo. En cualquier otro caso, retorna `False`.

3.1.3. 3. Análisis y Estadísticas de Transacciones (Listas)

- `calcular_balance_neto(transacciones)`: Recibe una lista de números (positivos para ingresos, negativos para egresos). Retorna la suma neta de todos los elementos. Si la lista está vacía, retorna 0.
- `filtrar_compras_mayores_a(transacciones, umbral)`: Extrae únicamente los egresos (valores negativos) cuyo valor absoluto supere el `umbral`. Por ejemplo, para `transacciones = [500, -120, -30, -250]` y `umbral = 100`, retornará `[-120, -250]`. Lanza un `ValueError` si el `umbral` provisto es negativo.

4. Estructura de Pruebas Requerida (test_finanzas.py)

Para la resolución guiada, estructuraremos al menos las siguientes pruebas demostrativas usando las mejores prácticas profesionales:

```
1 import pytest
2 from finanzas import deposito, retiro
3
4 # Ejemplo de caso normal
5 def test_deposito_exitoso():
6     """Verifica que un deposito valido incremente el saldo correctamente"""
7     saldo_nuevo = deposito(1000, 500)
8     assert saldo_nuevo == 1500
9
10 # Ejemplo de caso limite
11 def test_retiro_justo_al_limite():
12     """Verifica el retiro del 100% de los fondos disponibles."""
13     assert retiro(500, 500) == 0
14
```

```

15 # Ejemplo de validacion de excepciones
16 def test_retiro_fondos_insuficientes_error():
17     """Debe lanzar ValueError con el mensaje exacto de error."""
18     with pytest.raises(ValueError, match="Fondos insuficientes"):
19         retiro(100, 150)

```

Listing 1: Esqueleto base sugerido para test_finanzas.py

5. Comandos Clave para Ejecución en la Sesión

Durante la sesión en vivo utilizaremos la consola para validar nuestro código dinámicamente con los siguientes comandos:

- **Ejecución estándar:** `pytest -v`
- **Prueba de un método específico:** `pytest test_finanzas.py::test_deposito_exitoso`
- **Análisis de cobertura en terminal:** `pytest --cov=finanzas`
- **Generación de reporte interactivo HTML:** `pytest --cov=finanzas --cov-report=html`